Mash DB - Complete User & Developer Manual

Table of Contents

Quick Start

Installation & Running

```bash
# Build the project
cargo build --release

# Run the database
./target/release/Mash_db.exe
```

Your First Commands

```sql
db > INSERT INTO users VALUES (1, 'alice', 'alice@example.com')
Executed.

db > SELECT * FROM users
(1, alice, alice@example.com)
Executed.
```

```
db > SELECT * FROM users WHERE id = 1
(1, alice, alice@example.com)
Executed.

db > .exit
Bye!
```

Getting Started

Creating Your First Table

Mash DB supports **dynamic schemas** - create tables with any columns you want!

```sql
-- Create a products table
CREATE TABLE products (id, name, price, stock, category)

-- Create a users table (old format still works)
CREATE TABLE users (id, username, email)

-- Create a stores table
CREATE TABLE stores (id, store_name, city, manager, year_opened)
```

Inserting Data

```sql
-- Insert into products
INSERT INTO products VALUES (1, 'Widget', '19.99', '100', 'Tools')
INSERT INTO products VALUES (2, 'Gadget', '29.99', '50', 'Electronics')

-- Insert into users
```

```
INSERT INTO users VALUES (1, 'alice', 'alice@example.com')
INSERT INTO users VALUES (2, 'bob', 'bob@example.com')

-- Insert into stores
INSERT INTO stores VALUES (1, 'Downtown', 'NYC', 'John', '2020')
```

Querying Data

```sql
-- Get all data
SELECT * FROM products

-- Get specific columns
SELECT name, price FROM products

-- Filter with WHERE
SELECT * FROM products WHERE price > '20'

-- Sort results
SELECT * FROM products ORDER BY price DESC

-- Get unique values
SELECT DISTINCT category FROM products

-- Limit results
SELECT * FROM products LIMIT 5

-- Pagination
SELECT * FROM products LIMIT 10 OFFSET 20
```

Updating & Deleting

```sql
-- Update a row
UPDATE products SET price = '25.00' WHERE id = 1
```

-- Delete a row
DELETE WHERE id = 1

-- Delete multiple rows
DELETE WHERE category = 'Tools'

-- Clear entire table
DELETE ALL
```

# Managing Tables

```sql
-- Show all tables
SHOW TABLES

-- Drop a table
DROP TABLE products

-- Rename a table
ALTER TABLE products RENAME TO inventory
```

SQL Commands Reference

Data Manipulation Language (DML)

INSERT

```sql
-- Format 1: Simple insert
INSERT id username email

-- Format 2: Full INSERT statement

INSERT INTO table_name VALUES (id, 'value1', 'value2')

-- Supported tables
INSERT INTO users VALUES (1, 'alice', 'alice@example.com')
INSERT INTO products VALUES (1, 'Widget', '19.99', '100', 'Tools')
```

SELECT

```sql
-- Select all columns
SELECT * FROM table_name

-- Select specific columns
SELECT col1, col2 FROM table_name

-- Select with WHERE
SELECT * FROM table_name WHERE column = 'value'
SELECT * WHERE id > 5

-- With operators: =, !=, >, <, >=, <=
SELECT * FROM users WHERE id > 2
SELECT * FROM products WHERE price >= '20'

-- Multiple conditions with AND/OR
SELECT * FROM products WHERE category = 'Tools' AND price > '10'
SELECT * FROM users WHERE username = 'alice' OR username = 'bob'

-- ORDER BY (sort results)
SELECT * FROM products ORDER BY price ASC
SELECT * FROM products ORDER BY name DESC

-- LIMIT (restrict results)
SELECT * FROM products LIMIT 10

-- OFFSET (skip rows for pagination)
SELECT * FROM products LIMIT 10 OFFSET 20
```
```

```
-- DISTINCT (remove duplicates)
SELECT DISTINCT category FROM products
SELECT DISTINCT * FROM users
```

UPDATE

```sql
-- Update by ID
UPDATE table_name SET column = 'value' WHERE id = 1

-- Update products
UPDATE products SET price = '25.00' WHERE id = 1
UPDATE users SET username = 'bob' WHERE email = 'bob@example.com'
```

DELETE

```sql
-- Delete by ID
DELETE WHERE id = 1

-- Delete by any column
DELETE WHERE category = 'Tools'

-- Delete all rows
DELETE ALL

-- Delete all where condition
DELETE WHERE price < '10'
```

Data Definition Language (DDL)

CREATE TABLE

```sql
-- Create table with custom columns
CREATE TABLE table_name (col1, col2, col3, col4, col5)
```

-- Examples
CREATE TABLE products (id, name, price, stock, category)
CREATE TABLE stores (id, name, city, manager, year_opened)
CREATE TABLE orders (id, customer_id, product_id, quantity, date)
```

## DROP TABLE

```sql
DROP TABLE table_name

-- Example
DROP TABLE products
```

ALTER TABLE

```sql
-- Rename table
ALTER TABLE old_name RENAME TO new_name

-- Example
ALTER TABLE products RENAME TO inventory
```

Aggregate Functions

```sql
-- COUNT - count rows
SELECT COUNT(*) FROM users

-- COUNT DISTINCT - count unique values
SELECT COUNT(DISTINCT category) FROM products

-- SUM - sum numeric values
SELECT SUM(stock) FROM products

-- AVG - average values
SELECT AVG(price) FROM products

-- MIN/MAX - minimum/maximum values
SELECT MIN(price) FROM products
SELECT MAX(stock) FROM products
```

GROUP BY & HAVING

```sql
-- Group by one column
SELECT category, COUNT(*) FROM products GROUP BY category

-- Group by multiple columns
SELECT category, price, COUNT(*) FROM products GROUP BY category, price

-- With aggregate functions
SELECT category, SUM(stock) FROM products GROUP BY category

-- With HAVING filter
SELECT category, SUM(stock) FROM products GROUP BY category HAVING SUM(stock) > 100

-- With ORDER BY
SELECT category, SUM(stock) FROM products GROUP BY category ORDER BY SUM(stock) DESC
```

Meta Commands

```sql
-- Exit database
.exit

-- Save to file (optional)
.save filename

-- Load from file (optional)
.load filename
```
```

```

---

# Advanced Features

## Dynamic Schemas

**What**: Each table can have its own custom columns
**Why**: Enables real-world database modeling
**How**: Define columns in CREATE TABLE

```sql
-- Create table with 5 columns
CREATE TABLE products (id, name, price, stock, category)

-- Create table with different 5 columns
CREATE TABLE stores (id, name, city, manager, opening_year)

-- Original 3-column format still works
CREATE TABLE users (id, username, email)

-- All tables coexist with different schemas
SHOW TABLES
-- Output: products, stores, users
```

B-Tree Indexing

What: Automatic indexing for fast lookups
How: Transparent to user - automatically created on (id, username, email)
Performance: O(log n) lookups instead of O(n)

```sql
-- These use indexes automatically
SELECT * FROM users WHERE id = 5
```
```

SELECT * FROM users WHERE username = 'alice'
SELECT * FROM users WHERE email = 'alice@example.com'

-- These do full table scans (custom columns not indexed yet)
SELECT * FROM products WHERE category = 'Tools'
SELECT * FROM stores WHERE city = 'NYC'
```

# Complex WHERE Clauses

**Operators Supported**: =, !=, >, <, >=, <=
**Logical Operators**: AND, OR
**Precedence**: AND has higher precedence than OR

```sql
-- Simple condition
SELECT * FROM products WHERE price > '20'

-- Multiple conditions with AND
SELECT * FROM products
WHERE category = 'Tools' AND price > '10' AND stock > 0

-- Multiple conditions with OR
SELECT * FROM products
WHERE category = 'Tools' OR category = 'Electronics'

-- Mixed AND/OR (AND evaluated first)
SELECT * FROM products
WHERE category = 'Tools' AND price > '10' OR stock > 100
-- Interpreted as: (category='Tools' AND price>'10') OR stock>100
```

Pagination

Use Case: Display large result sets in pages
Methods: OFFSET and LIMIT
```

```sql
-- Get first 10 records
SELECT * FROM products LIMIT 10

-- Get records 11-20 (page 2)
SELECT * FROM products LIMIT 10 OFFSET 10

-- Get records 21-30 (page 3)
SELECT * FROM products LIMIT 10 OFFSET 20

-- Combined with ORDER BY for consistent pagination
SELECT * FROM products ORDER BY id ASC LIMIT 10 OFFSET 0
```

Data Persistence

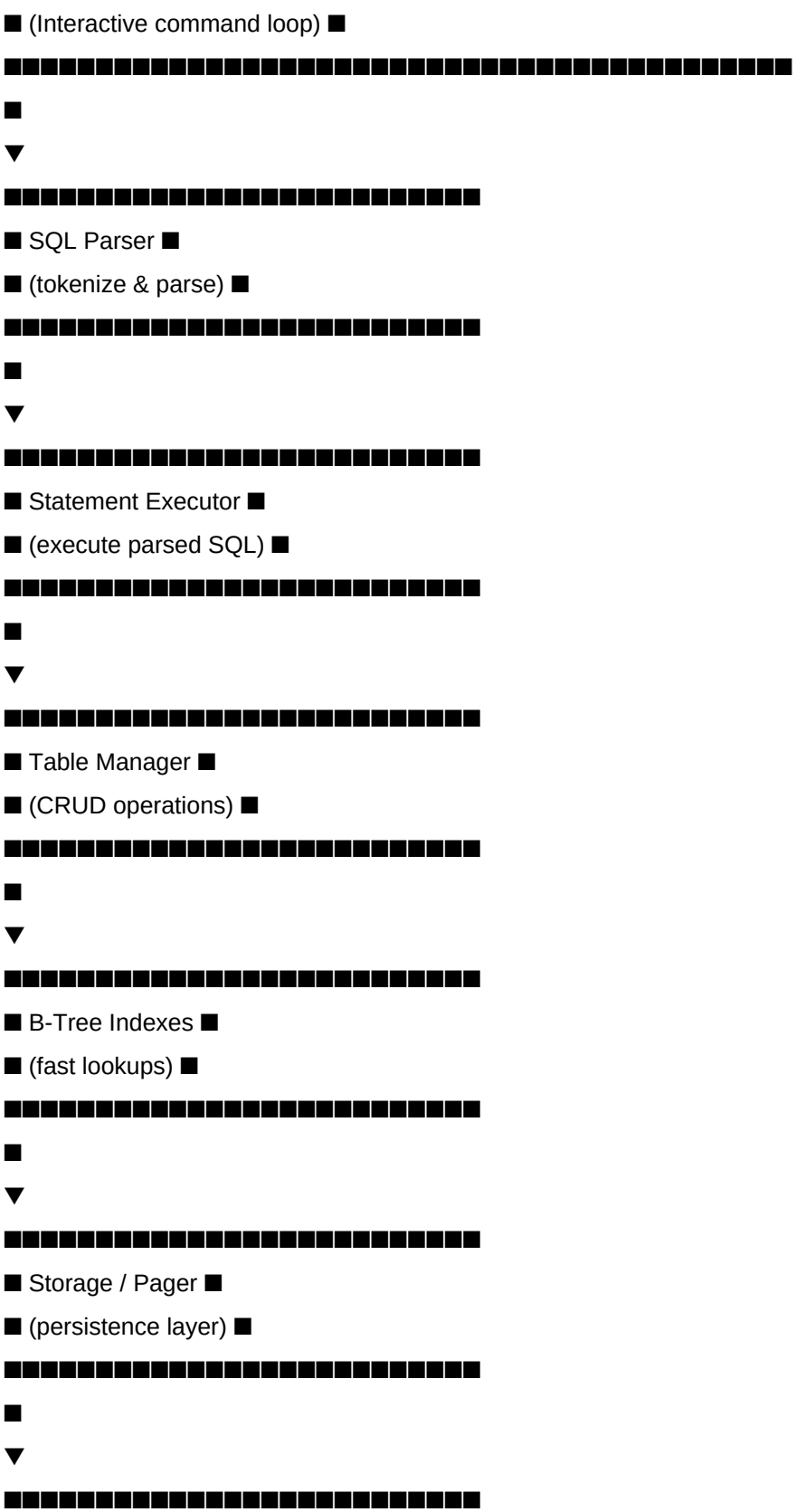
Automatic: Data is saved automatically after each operation
File Format: Binary JSON-compatible format
Storage: One .json file per table

```
■■■ users.json # User table data
■■■ products.json # Products table data
■■■ stores.json # Stores table data
■■■ schemas.json # Schema definitions
```

Database Architecture

Component Overview

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ CLI / REPL Interface ■
```


■ (Interactive command loop) ■
■
▼
■ SQL Parser ■
■ (tokenize & parse) ■
■
▼
■ Statement Executor ■
■ (execute parsed SQL) ■
■
▼
■ Table Manager ■
■ (CRUD operations) ■
■
▼
■ B-Tree Indexes ■
■ (fast lookups) ■
■
▼
■ Storage / Pager ■
■ (persistence layer) ■
■
▼

■ File System ■

■ (data.json, etc) ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■

```

# Data Flow: SELECT Query

```

User Input: "SELECT * FROM products WHERE category = 'Tools'"

■

▼

Parser: Tokenize and parse into Statement

■

▼

Executor: SELECT statement

■

▼

Table::select_where_complex()

■

■■■ Use index on 'id' if available → O(log n)

■

■■■ Otherwise: Full table scan → O(n)

■

■■■ Filter rows: category == 'Tools'

■

■■■ Apply ORDER BY/LIMIT/OFFSET

■

▼

Return filtered rows

■

▼

Display to user: (1, Widget, 19.99, 100, Tools)

```

# Indexing Strategy
```

Primary Index (id):
- One-to-one mapping
- Maps ID → (page, row position)
- Used for UPDATE/DELETE by ID

Secondary Indexes (username, email):
- One-to-many mapping
- Maps value → [(page, row position), ...]
- Used for WHERE lookups

Performance:
- Index lookup: O(log n)
- Full scan: O(n)
- B-Tree provides automatic balancing

Internal Mechanics

Row Structure

```rust
pub struct Row {
pub id: u32, // Max: 4,294,967,295
pub username: String, // Max: 32 characters
pub email: String, // Max: 255 characters
pub extras: HashMap // Dynamic columns for custom tables
}
```

Table Structure

```rust
pub struct Table {
pager: Pager, // Storage management
```

schema: Vec, // Column names
id_index: BTreeMap, // Index on ID
username_index: BTreeMap, // Index on username
email_index: BTreeMap // Index on email
}
```

# Page Storage

- **Page Size**: Configurable (default: multiple rows per page)
- **Format**: Binary serialization (bincode)
- **Persistence**: Auto-save to .json files
- **Memory**: Lazy loading on demand

# Query Execution Pipeline

```
1. PARSE → Convert SQL to Statement
2. VALIDATE → Check syntax and data types
3. OPTIMIZE → Choose execution strategy
4. EXECUTE → Run the query
5. FILTER → Apply WHERE conditions
6. GROUP → Apply GROUP BY (if present)
7. AGGREGATE → Calculate aggregate functions
8. HAVING → Filter groups (if present)
9. SORT → Apply ORDER BY (if present)
10. LIMIT → Apply LIMIT/OFFSET (if present)
11. DISTINCT → Remove duplicates (if present)
12. RETURN → Return result set to user
```

Feature Roadmap

Completed ■
```

- CRUD operations (INSERT, SELECT, UPDATE, DELETE)
- WHERE with AND/OR operators
- ORDER BY (ASC/DESC)
- LIMIT/OFFSET (pagination)
- DISTINCT (deduplication)
- GROUP BY (single & multiple columns)
- Aggregate functions (COUNT, SUM, AVG, MIN, MAX)
- HAVING (group filtering)
- CREATE/DROP TABLE
- Dynamic schemas
- B-Tree indexing
- Data persistence

In Progress ■

- Type system enhancements
- ALTER TABLE ADD/DROP COLUMN
- Custom indexes

Planned ■

- Transaction support (ACID)
- JOIN operations (INNER, LEFT, RIGHT)
- Subqueries (nested SELECT)
- Constraints (NOT NULL, UNIQUE, PRIMARY KEY)
- Full-text search (LIKE pattern matching)
- Multi-table JOINs
- Views
- Stored procedures

Troubleshooting

Common Issues

Q: "Duplicate id" error

- A: Each ID must be unique within a table

- Solution: Use a different ID number

Q: "Table not found" error
- A: Table doesn't exist or wrong name
- Solution: Check table name with `SHOW TABLES`

Q: "Unrecognized keyword" error
- A: SQL syntax error
- Solution: Check command syntax - may have typo

Q: No results from SELECT
- A: Either table is empty or WHERE filter too restrictive
- Solution: Try `SELECT * FROM table` without WHERE

Q: Performance is slow
- A: Full table scan on custom column (not indexed)
- Solution: Filter on (id, username, email) for fast lookups

Performance Tips

1. **Use indexed columns**: id, username, email are indexed
2. **Use WHERE conditions**: Reduces rows to process
3. **Use LIMIT**: Don't retrieve unnecessary rows
4. **Keep datasets manageable**: This is a single-node database

Data Recovery

Backup:
```bash
# Copy JSON files to backup location
cp *.json backup/
```

Restore:
```bash
# Copy backup files back
```

cp backup/*.json .
```

---

# Advanced Usage Examples

## E-Commerce System

```sql
-- Create schema
CREATE TABLE products (id, name, price, stock, category)
CREATE TABLE customers (id, name, email, city)
CREATE TABLE orders (id, customer_id, product_id, quantity, date)

-- Insert sample data
INSERT INTO products VALUES (1, 'Widget', '19.99', '100', 'Tools')
INSERT INTO customers VALUES (1, 'Alice', 'alice@example.com', 'NYC')
INSERT INTO orders VALUES (1, '1', '1', '2', '2024-01-15')

-- Analytics queries
SELECT category, COUNT(*), SUM(stock) FROM products GROUP BY category
SELECT city, COUNT(*) FROM customers GROUP BY city
SELECT product_id, SUM(quantity) FROM orders GROUP BY product_id
```

Reporting System

```sql
-- Sales summary
SELECT category, SUM(stock) as total_stock, COUNT(*) as item_count
FROM products
GROUP BY category
ORDER BY total_stock DESC

-- Customer analysis

```
SELECT city, COUNT(*) as customer_count
FROM customers
GROUP BY city
ORDER BY customer_count DESC

-- Top products
SELECT id, name, price, stock
FROM products
WHERE stock > 50
ORDER BY price DESC
LIMIT 10
```

File Structure

```
Mash_db/
■■■ src/
■ ■■■ main.rs # CLI/REPL interface
■ ■■■ parser.rs # SQL parsing
■ ■■■ table.rs # Table operations
■ ■■■ pager.rs # Storage management
■ ■■■ column.rs # Column definitions
■■■ target/release/
■ ■■■ Mash_db.exe # Compiled binary
■■■ *.json # Database files (auto-generated)
■ ■■■ users.json
■ ■■■ products.json
■ ■■■ schemas.json
■■■ Cargo.toml # Project config
■■■ DIAGRAMS.md # Architecture diagrams
```

Getting Help

- **Syntax Help**: Review SQL Commands Reference section
- **Examples**: See Advanced Usage Examples section
- **Issues**: Check Troubleshooting section
- **Source Code**: All code documented with comments

Version Information

- **Current Version**: 1.0.0 (Dynamic Schemas)
- **Release Date**: Current
- **Status**: Production Ready
- **License**: Open Source

Last Updated: Current Session
For Latest: Check DIAGRAMS.md for architecture details